



Configuración de Seguridad Spring Boot usando JPA y Base de Datos

Introducción

En esta guía vamos a aprender a **modificar la configuración de seguridad** de nuestra aplicación Spring Boot, de modo que ya **no utilice usuarios en memoria**, sino que ahora **los recupere desde una base de datos** utilizando JPA.

Durante la guía trabajaremos en:

- ✱ Eliminar la autenticación en memoria.
- ✱ Inyectar el servicio de usuarios personalizados.
- ✱ Definir un `PasswordEncoder` usando BCrypt.
- ✱ Configurar la seguridad para cargar usuarios de base de datos.
- ✱ Agregar la anotación `@Transactional` en el servicio de usuarios.
- ✱ Probar el login desde el navegador usando usuarios registrados en la base de datos.

Desarrollo Paso a Paso

1. Modificar la clase `SecurityConfig.java`

Ruta del archivo:

`src/main/java/gm/seguridad/SecurityConfig.java`

Código trabajado paso a paso:

Primero, **eliminamos** el método anterior donde creábamos usuarios con `userDetailsService()`.

Después, **inyectamos el servicio** `UserDetailsService` que hemos creado previamente en el constructor de la clase:

```
private final UserDetailsService userDetailsService;

public SecurityConfig(UserDetailsService userDetailsService) {
    this.userDetailsService = userDetailsService;
}
```

✦ Definición del proveedor de autenticación

```
@Bean
public DaoAuthenticationProvider authenticationProvider() {
    DaoAuthenticationProvider authProvider = new DaoAuthenticationProvider();
    authProvider.setUserDetailsService(userDetailsService);
    authProvider.setPasswordEncoder(passwordEncoder()); // Aquí llamamos al método, no al atributo
    return authProvider;
}
```

- Se crea un `DaoAuthenticationProvider` que:
 - Usa el `UserDetailsService` inyectado para autenticar usuarios.
 - Usa `BCryptPasswordEncoder` para comparar contraseñas encriptadas.

✦ Definición del encriptador de contraseñas

```
@Bean
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}
```

- Se crea un `PasswordEncoder` utilizando el algoritmo seguro `BCrypt`.

Con esto ya tenemos completa nuestra clase SecurityConfig

2. Modificar el servicio UsuarioService.java

Ruta del archivo:

src/main/java/gm/servicio/UsuarioService.java

Código trabajado paso a paso:

Agregamos la anotación `@Transactional(readonly = true)` en el método `loadUserByUsername` para indicar que este método trabaja en modo de **solo lectura** sobre la base de datos, lo que evita errores de sesión de Hibernate y mejora el rendimiento:

```
@Override
@Transactional(readonly=true)
public UserDetails loadUserByUsername(String username) throws UsernameNotFoundException {
    Usuario usuario = usuarioDao.findByUsername(username);

    if(usuario == null){
        throw new UsernameNotFoundException(username);
    }

    var roles = new ArrayList<GrantedAuthority>();

    for(Rol rol: usuario.getRoles()){
        roles.add(new SimpleGrantedAuthority(rol.getNombre()));
    }

    return new User(usuario.getUsername(), usuario.getPassword(), roles);
}
```

👉 Gracias a esta anotación, evitamos errores como "no session found" al intentar cargar roles asociados a un usuario.

Código Final Completo de cada archivo trabajado

SecurityConfig.java

```
package gm.web;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
```

```
import org.springframework.security.authentication.dao.DaoAuthenticationProvider;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
public class SecurityConfig {

    private final UserDetailsService userDetailsService;

    public SecurityConfig(UserDetailsService userDetailsService) {
        this.userDetailsService = userDetailsService;
    }

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        return http
            .authorizeHttpRequests(authorize -> authorize
                .requestMatchers("/login", "/resources/**").permitAll()
                .requestMatchers("/editar/**", "/agregar/**", "/eliminar/**").hasRole("ADMIN")
                .requestMatchers("/").hasAnyRole("USER", "ADMIN")
                .anyRequest().authenticated()
            )
            .formLogin(form -> form
                .loginPage("/login")
                .defaultSuccessUrl("/", true)
                .permitAll()
            )
            .logout(logout -> logout
                .logoutUrl("/logout")
                .logoutSuccessUrl("/login?logout")
                .invalidateHttpSession(true)
                .deleteCookies("JSESSIONID")
            )
            .exceptionHandling(exception -> exception
                .accessDeniedPage("/errores/403")
            )
            .build();
    }

    @Bean
    public DaoAuthenticationProvider authenticationProvider() {
        DaoAuthenticationProvider authProvider = new DaoAuthenticationProvider();
    }
}
```

```
    authProvider.setUserDetailsService(userDetailsService);
    authProvider.setPasswordEncoder(passwordEncoder()); // Aquí llamamos al método, no al atributo
    return authProvider;
}

@Bean
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}
}
```

✅ UsuarioService.java

```
package gm.servicio;

import java.util.ArrayList;
import gm.dao.UsuarioDao;
import gm.domain.Rol;
import gm.domain.Usuario;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.security.core.GrantedAuthority;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.security.core.userdetails.User;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.core.userdetails.UsernameNotFoundException;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

@Service("userService")
public class UsuarioService implements UserDetailsService{

    @Autowired
    private UsuarioDao usuarioDao;

    @Override
    @Transactional(readOnly=true)
    public UserDetails loadUserByUsername(String username) throws UsernameNotFoundException {
        Usuario usuario = usuarioDao.findByUsername(username);

        if(usuario == null){
            throw new UsernameNotFoundException(username);
        }
    }
}
```

```
var roles = new ArrayList<GrantedAuthority>();

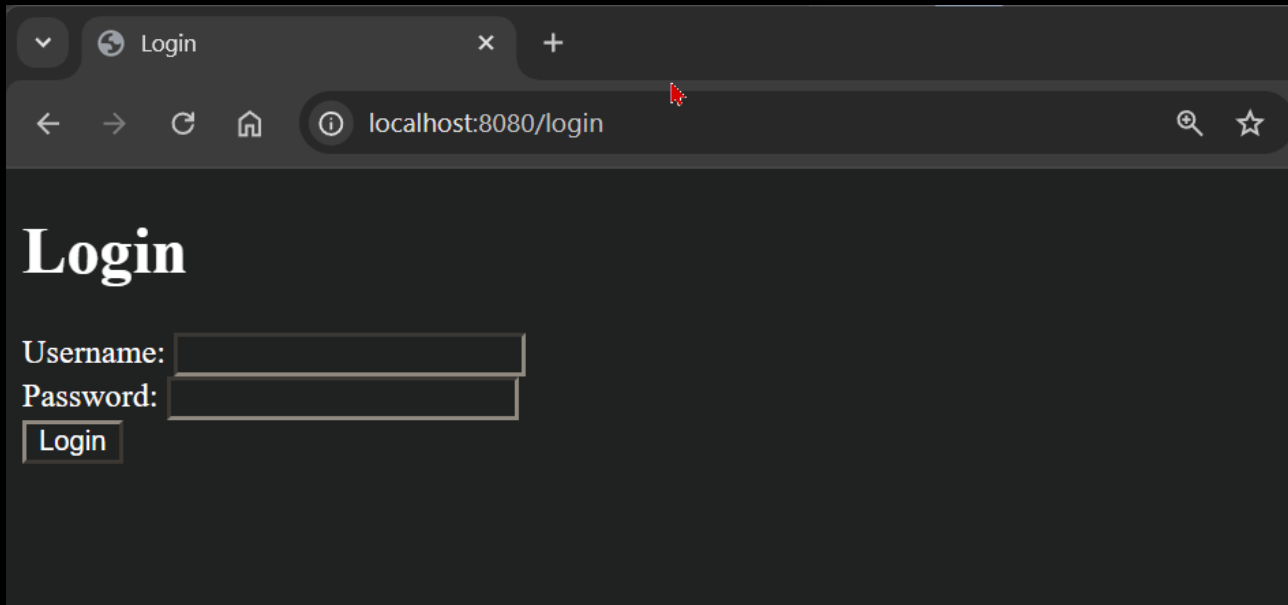
for(Rol rol: usuario.getRoles()){
    roles.add(new SimpleGrantedAuthority(rol.getNombre()));
}

return new User(usuario.getUsername(), usuario.getPassword(), roles);
}
```

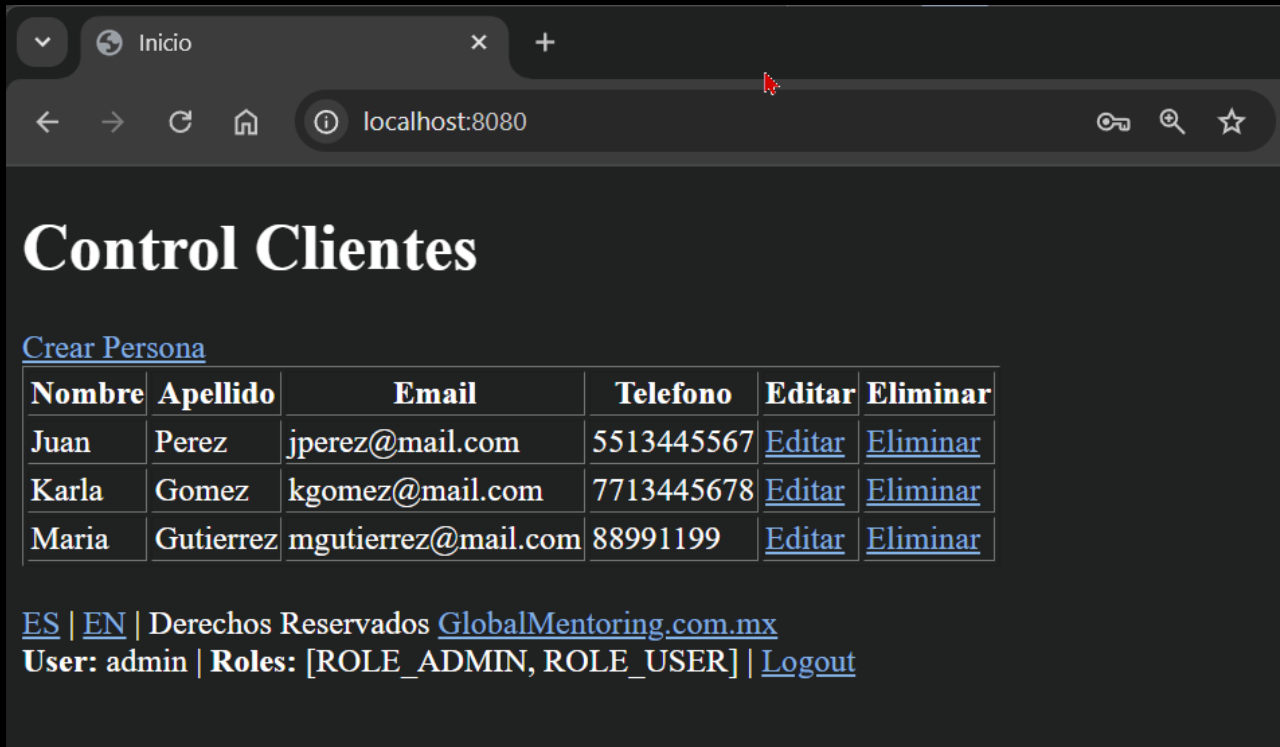
3. Probar el Proyecto 🚀

Cuando ya tenemos todo configurado:

- Accedemos a la app en el navegador (`localhost:8080`).
- Se mostrará el formulario de login.
- Ingresamos con un usuario válido de la base de datos.
- Spring cargará los roles desde la base de datos y aplicará las restricciones de acceso configuradas.



The screenshot shows a web browser window with a single tab titled 'Login'. The address bar displays 'localhost:8080/login'. The page content features a large 'Login' heading, followed by 'Username:' and 'Password:' labels next to text input fields. A 'Login' button is positioned at the bottom left of the form area.



Control Clientes

[Crear Persona](#)

Nombre	Apellido	Email	Telefono	Editar	Eliminar
Juan	Perez	jperez@mail.com	5513445567	Editar	Eliminar
Karla	Gomez	kgomez@mail.com	7713445678	Editar	Eliminar
Maria	Gutierrez	mgutierrez@mail.com	88991199	Editar	Eliminar

[ES](#) | [EN](#) | Derechos Reservados [GlobalMentoring.com.mx](#)
User: admin | Roles: [ROLE_ADMIN, ROLE_USER] | [Logout](#)

Conclusión

En esta lección hemos aprendido a:

- ✓ Inyectar un `UserDetailsService` personalizado.
- ✓ Configurar `BCryptPasswordEncoder` para proteger contraseñas.
- ✓ Cargar usuarios y roles directamente desde base de datos usando JPA.
- ✓ Hacer transaccional el método de carga de usuarios para evitar errores de sesión.
- ✓ Aplicar toda la seguridad de Spring Boot de forma profesional, eliminando usuarios en memoria.

Esto es muy importante porque ahora nuestra aplicación trabaja de manera más segura, escalable y lista para crecer. ¡Excelente avance! 

Sigue adelante con tu aprendizaje  , ¡el esfuerzo vale la pena!

¡Saludos! 

Ing. Marcela Gamiño e Ing. Ubaldo Acosta

Fundadores de [GlobalMentoring.com.mx](#)